
Interface Utilisateur : Clavier RFID et Codes d'Accès

Système de contrôle d'accès sécurisé par code PIN et verrou électronique

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

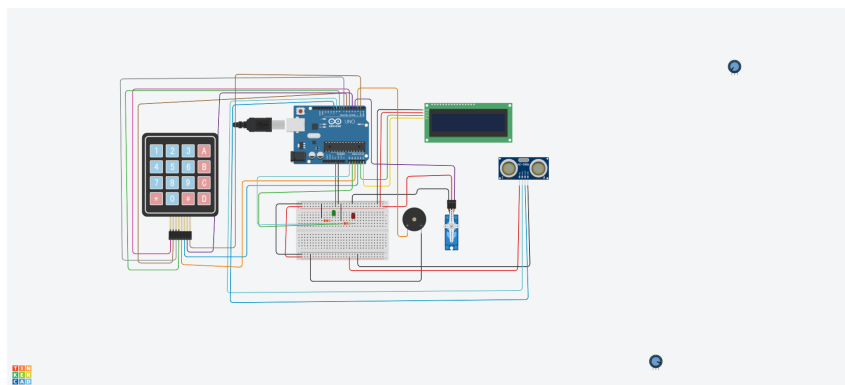


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges (version complète)	2
3.2	Composants utilisés dans la simulation Tinkercad	3
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	9
8	Ressources Complémentaires	10
9	Conclusion	10

1 Introduction

Ce rapport présente un **système de contrôle d'accès sécurisé** basé sur la carte **Arduino Uno**, combinant initialement un lecteur **RFID RC522** (lecture de badges à 13,56 MHz) et un **clavier matriciel 4x4** pour la saisie d'un code PIN. L'accès est accordé ou refusé en comparant la saisie de l'utilisateur à une base de données stockée en **mémoire EEPROM**, un **servomoteur** simulant physiquement le verrou électronique. L'historique des accès est destiné à être affiché sur un écran OLED.

Le module RFID RC522 n'étant pas disponible nativement dans la bibliothèque de composants **Tinkercad**, la version simulée de ce projet repose exclusivement sur la **saisie de code par clavier matriciel**, complétée par un **capteur ultrason** jouant le rôle de détecteur de présence (déclenchant l'invite de saisie lorsqu'un utilisateur s'approche), et un écran **LCD I²C** en remplacement de l'OLED. Cette version permet de valider entièrement la logique de contrôle d'accès (saisie, vérification, verrouillage/déverrouillage, EEPROM) indépendamment de la technologie RFID.

2 Description du Projet

Le système de contrôle d'accès repose sur la séquence fonctionnelle suivante :

- **détection de présence** : dans la version complète, l'approche d'un badge RFID active le lecteur ; dans la simulation, un **capteur ultrason** détecte la proximité d'un utilisateur et déclenche l'invite de saisie sur l'écran ;
- **saisie du code** : l'utilisateur compose un code à 4 chiffres sur le **clavier matriciel 4x4**, chaque chiffre saisi étant masqué à l'écran par un astérisque ;
- **vérification** : le code saisi est comparé au code de référence stocké en **mémoire EEPROM** (mémoire non volatile conservant les données même après coupure d'alimentation) ;
- **décision d'accès** : en cas de correspondance, le **servomoteur** pivote pour simuler le déverrouillage, accompagné d'une LED verte et d'un bip de confirmation ; en cas d'échec, une LED rouge et une séquence de bips d'alerte signalent le refus d'accès.

Dans la version cible du projet, chaque tentative d'accès (autorisée ou refusée) serait également journalisée et affichée sous forme d'**historique** sur l'écran OLED, une fonctionnalité qui pourrait être ajoutée à la structure logicielle existante en complément de l'affichage en temps réel.

Objectif pédagogique : Ce projet met en pratique la gestion d'une interface de saisie utilisateur (clavier matriciel), la persistance de données sensibles en mémoire non volatile (EEPROM), ainsi que le pilotage d'un actionneur mécanique (servomoteur) en réponse à une décision logicielle – une architecture caractéristique des systèmes de sécurité embarqués.

3 Composants Utilisés

3.1 Composants du cahier des charges (version complète)

lightgray Composant	Rôle dans le projet
Module RFID RC522	Lecture de cartes/badges sans contact à 13,56 MHz (communication SPI)
Clavier 4x4	Saisie manuelle d'un code PIN en alternative ou complément au badge
Servo SG90	Actionneur simulant le verrou électronique (ouverture/-fermeture)
Écran OLED 0,96"	Affichage de l'état du système et de l'historique des accès
Buzzer	Signal sonore de confirmation ou d'alerte
LEDs	Indicateurs visuels d'état (accès autorisé/refusé)
Arduino Uno	Microcontrôleur assurant la lecture des entrées, la logique d'accès et le pilotage des sorties

TABLE 1 – Composants prévus dans le cahier des charges initial du projet.

lightgray Réf- rence	Quantité	Composant
U1	1	Arduino Uno R3
PIEZO2	1	Buzzer piézoélectrique (signal sonore de confirmation/alerte)
U2	1	Écran LCD 16x2 avec module I ² C basé sur le PCF8574 (adresse 0x20)
SERVO1	1	Servomoteur positionnel (simulation du verrou électronique)
D1	1	LED rouge (accès refusé / système verrouillé)
D2	1	LED verte (accès autorisé / déverrouillage)
DIST2	1	Capteur de distance ultrasonique (détection de présence, remplace l'approche du badge RFID)
R2, R3	2	Résistances 220 Ω (limitation de courant des LED)
KEYPAD1	1	Clavier matriciel 4x4 (saisie du code PIN)

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

3.2 Composants utilisés dans la simulation Tinkercad

Remarque : Le module **RFID RC522** n'étant pas proposé par Tinkercad, la simulation remplace la lecture de badge par une **détection de proximité par ultrason**, qui joue un rôle fonctionnel analogue de déclencheur (présence détectée → invite de saisie). L'écran **OLED** est également remplacé par un **LCD I²C**, plus simple à simuler. La logique de vérification de code, de stockage EEPROM et de pilotage du servomoteur reste en revanche strictement identique à la version cible.

4 Notions Électroniques Clés

- **Communication SPI (MOSI, MISO, SCK, SS) – vitesse, modes :** le module RFID RC522 communique avec l'Arduino via le bus **SPI** (Serial Peripheral Interface), qui utilise quatre lignes (**MOSI** pour les données maître→esclave, **MISO** pour esclave→maître, **SCK** pour l'horloge, **SS/CS** pour la sélection du périphérique), offrant des débits nettement supérieurs à l'I²C grâce à l'absence d'adressage et à une horloge dédiée.

- **Principe de la RFID (couplage inductif, ISO 14443-A)** : un badge RFID passif ne contient aucune source d'énergie propre ; il est alimenté par **couplage inductif** avec l'antenne du lecteur, qui génère un champ électromagnétique à 13,56 MHz. Le standard **ISO 14443-A** définit le protocole de communication entre le lecteur et la carte, incluant l'anticollision (gestion de plusieurs badges simultanés) et le transfert de l'identifiant unique (UID).
- **Mémoire EEPROM interne (lecture/écriture, durée de vie)** : l'Arduino Uno intègre 1 Ko d'**EEPROM** (Electrically Erasable Programmable Read-Only Memory), une mémoire non volatile permettant de conserver des données (ici, le code PIN de référence) même après coupure d'alimentation. Cette mémoire supporte un nombre limité de cycles d'écriture (typiquement environ 100 000 cycles par cellule), ce qui impose de limiter les écritures fréquentes dans une application réelle.
- **Chiffrement simple des UUIDs (XOR, hash)** : pour éviter de stocker en clair l'identifiant d'un badge autorisé, celui-ci peut être transformé par une opération simple (par exemple un **XOR** avec une clé secrète, ou une fonction de **hachage** légère), rendant plus difficile la duplication d'un accès à partir d'une simple lecture de la mémoire du microcontrôleur.

5 Schéma du Montage

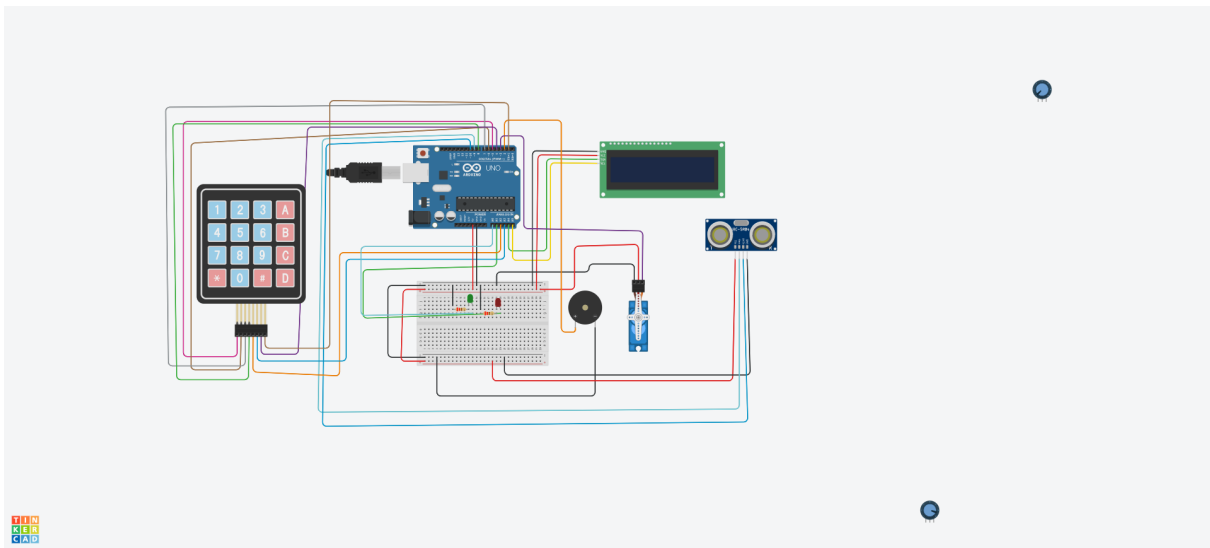


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, clavier 4x4, LCD I²C, servomoteur, ultrason, LED, buzzer).

6 Code Arduino

Le code ci-dessous a été généré pour reproduire fidèlement la logique du cahier des charges avec les composants disponibles sous Tinkercad : détection de présence par ultrason, saisie d'un code à 4 chiffres au clavier matriciel, vérification contre une valeur de référence stockée en EEPROM, et pilotage du servomoteur, du buzzer et des LED selon le résultat de la vérification.

```

1 #include <Keypad.h>
2 #include <LiquidCrystal_I2C.h>
3 #include <Servo.h>
4 #include <EEPROM.h>

```

```

5
6 // ===== ECRAN LCD I2C =====
7 LiquidCrystal_I2C lcd(0x20, 16, 2);
8
9 // ===== CLAVIER MATRICIEL 4x4 =====
10 const byte LIGNES = 4;
11 const byte COLONNES = 4;
12
13 char touches[LIGNES][COLONNES] = {
14     {'1','2','3','A'},
15     {'4','5','6','B'},
16     {'7','8','9','C'},
17     {'*','0','#','D'}
18 };
19
20 byte brochesLignes[LIGNES] = {9, 8, 7, 6}; // R1, R2, R3, R4
21 byte brochesColonnes[COLONNES] = {5, 4, 3, A3}; // C1, C2, C3, C4
22
23 Keypad clavier = Keypad(makeKeymap(touches), brochesLignes, brochesColonnes,
24     LIGNES, COLONNES);
25
26 // ===== SERVOMOTEUR (VERROU) =====
27 Servo servoVerrou;
28 const int pinServo = 10;
29 const int angleFerme = 0;
30 const int angleOuvert = 90;
31
32 // ===== BUZZER ET LEDs =====
33 const int pinBuzzer = 11;
34 const int pinLedRouge = 12;
35 const int pinLedVerte = 13;
36
37 // ===== CAPTEUR ULTRASON (DETECTION DE PRESENCE)
38 // =====
39 const int pinTrigger = A0;
40 const int pinEcho = A1;
41 const int seuilDetectionCm = 15; // Distance en dessous de laquelle on
42     consid re une pr sence
43
44 // ===== GESTION DU CODE PIN EN EEPROM =====
45 // Adresses EEPROM 0 3 : code PIN 4 chiffres (un octet par chiffre)
46 const int adresseEEPROM = 0;
47 const byte tailleCode = 4;
48 char codeSaisi[tailleCode + 1]; // +1 pour le caract re de fin de cha ne
49 byte indexSaisie = 0;
50
51 // Code PIN par d faut , crit en EEPROM uniquement si elle est vierge (
52     premi re utilisation)
53 char codeParDefaut[tailleCode + 1] = "1234";
54
55 void setup() {
56     Serial.begin(9600);
57
58     // --- Initialisation LCD ---
59     lcd.init();
60     lcd.backlight();
61     lcd.setCursor(0, 0);
62     lcd.print("CONTROLE ACCES");
63     lcd.setCursor(0, 1);
64     lcd.print("Initialisation..");
65     delay(1500);
66
67     // --- Initialisation Servo (verrou ferm par d faut) ---

```

```

64  servoVerrou.attach(pinServo);
65  servoVerrou.write(angleFerme);
66
67  // --- Initialisation des sorties ---
68  pinMode(pinBuzzer, OUTPUT);
69  pinMode(pinLedRouge, OUTPUT);
70  pinMode(pinLedVerte, OUTPUT);
71  pinMode(pinTrigger, OUTPUT);
72  pinMode(pinEcho, INPUT);
73
74  digitalWrite(pinLedRouge, HIGH); // LED rouge = syst me verrouill /au repos
75  digitalWrite(pinLedVerte, LOW);
76
77  // --- Initialisation du code PIN en EEPROM (si premi re utilisation) ---
78  initialiserCodeEEPROM();
79
80  afficherEcranAttente();
81 }
82
83 void loop() {
84  // 1. D tecton de pr sence par ultrason (simule l'approche d'un badge/
      utilisateur)
85  long distance = mesurerDistanceCm();
86
87  if (distance > 0 && distance < seuilDetectionCm) {
88      demarrerSaisieCode();
89  }
90
91  // 2. Lecture du clavier si une saisie est en cours
92  char touche = clavier.getKey();
93  if (touche) {
94      gererTouche(touche);
95  }
96 }
97
98 // =====
99 //                FONCTIONS DE MESURE ULTRASON
100 // =====
101 long mesurerDistanceCm() {
102     digitalWrite(pinTrigger, LOW);
103     delayMicroseconds(2);
104     digitalWrite(pinTrigger, HIGH);
105     delayMicroseconds(10);
106     digitalWrite(pinTrigger, LOW);
107
108     long duree = pulseIn(pinEcho, HIGH, 30000); // timeout 30 ms
109     long distance = duree * 0.034 / 2; // vitesse du son ~340 m/s
110     return distance;
111 }
112
113 // =====
114 //                GESTION DE LA SAISIE DU CODE
115 // =====
116 void demarrerSaisieCode() {
117     lcd.clear();
118     lcd.setCursor(0, 0);
119     lcd.print("Entrez le code:");
120     lcd.setCursor(0, 1);
121     indexSaisie = 0;
122     memset(codeSaisi, 0, sizeof(codeSaisi));
123
124     bip(50); // 1 ger bip pour signaler la d tecton de pr sence
125 }

```

```

126
127 void gererTouche(char touche) {
128     // Touche '*' : annuler la saisie en cours
129     if (touche == '*') {
130         afficherEcranAttente();
131         return;
132     }
133
134     // Touche '#' : valider le code saisi
135     if (touche == '#') {
136         verifierCode();
137         return;
138     }
139
140     // Sinon, on enregistre le chiffre saisi (dans la limite de 4 chiffres)
141     if (indexSaisie < tailleCode) {
142         codeSaisi[indexSaisie] = touche;
143         indexSaisie++;
144         lcd.setCursor(indexSaisie - 1, 1);
145         lcd.print('*'); // masquage du chiffre saisi    1' cran
146         bip(20);
147     }
148 }
149
150 // =====
151 //                VERIFICATION DU CODE ET PILOTAGE DU VERROU
152 // =====
153 void verifierCode() {
154     char codeEnregistre[tailleCode + 1];
155     lireCodeEEPROM(codeEnregistre);
156
157     if (strncmp(codeSaisi, codeEnregistre, tailleCode) == 0) {
158         accesAutorise();
159     } else {
160         accesRefuse();
161     }
162 }
163
164 void accesAutorise() {
165     lcd.clear();
166     lcd.setCursor(0, 0);
167     lcd.print("ACCES AUTORISE");
168
169     digitalWrite(pinLedRouge, LOW);
170     digitalWrite(pinLedVerte, HIGH);
171     bip(100);
172
173     servoVerrou.write(angleOuvert); // d verrouillage
174     delay(3000);                    // maintien ouvert 3 secondes
175     servoVerrou.write(angleFerme);  // reverrouillage automatique
176
177     digitalWrite(pinLedVerte, LOW);
178     digitalWrite(pinLedRouge, HIGH);
179
180     afficherEcranAttente();
181 }
182
183 void accesRefuse() {
184     lcd.clear();
185     lcd.setCursor(0, 0);
186     lcd.print("ACCES REFUSE");
187
188     // Triple bip d'alerte

```

```

189   for (int i = 0; i < 3; i++) {
190       bip(150);
191       delay(150);
192   }
193
194   delay(1000);
195   afficherEcranAttente();
196 }
197
198 void afficherEcranAttente() {
199     lcd.clear();
200     lcd.setCursor(0, 0);
201     lcd.print("Approchez-vous");
202     lcd.setCursor(0, 1);
203     lcd.print("pour entrer code");
204     indexSaisie = 0;
205 }
206
207 // =====
208 //                      GESTION DU BUZZER
209 // =====
210 void bip(int dureeMs) {
211     digitalWrite(pinBuzzer, HIGH);
212     delay(dureeMs);
213     digitalWrite(pinBuzzer, LOW);
214 }
215
216 // =====
217 //                      GESTION DU CODE PIN EN EEPROM
218 // =====
219 void initialiserCodeEEPROM() {
220     // V rifie si l'EEPROM contient d j    un code valide (caract res
221     num riques)
222     bool codeValide = true;
223     for (int i = 0; i < tailleCode; i++) {
224         char c = EEPROM.read(adresseEEPROM + i);
225         if (c < '0' || c > '9') {
226             codeValide = false;
227             break;
228         }
229     }
230
231     // Si aucun code valide n'est trouv , on crit le code par d faut
232     if (!codeValide) {
233         for (int i = 0; i < tailleCode; i++) {
234             EEPROM.write(adresseEEPROM + i, codeParDefaut[i]);
235         }
236     }
237 }
238
239 void lireCodeEEPROM(char* destination) {
240     for (int i = 0; i < tailleCode; i++) {
241         destination[i] = EEPROM.read(adresseEEPROM + i);
242     }
243     destination[tailleCode] = '\0';
244 }

```

Listing 1 – Code source Arduino du système de contrôle d'accès par code PIN (simulation Tinkercad).

7 Analyse et Discussion du Code

Le code peut être décomposé en six blocs fonctionnels principaux :

1. **Déclarations et initialisation (setup)** : le LCD I²C, le servomoteur (positionné en *fermé* par défaut), les sorties (LED, buzzer) et les broches du capteur ultrason sont initialisés. La fonction `initialiserCodeEEPROM()` vérifie si un code valide est déjà présent en EEPROM et, à défaut, y écrit un code par défaut ("1234"), garantissant un fonctionnement correct dès la première mise sous tension.
2. **Détection de présence (mesurerDistanceCm)** : à chaque itération de la boucle principale, la distance est mesurée par la méthode classique d'émission/réception d'écho ultrasonique (`pulseIn` avec *timeout* de 30 ms pour éviter un blocage en l'absence d'écho). Si la distance mesurée passe sous le seuil défini (15 cm), la saisie du code est automatiquement déclenchée – ce mécanisme remplace ici la détection d'approche d'un badge RFID.
3. **Saisie du code (demarrerSaisieCode, gererTouche)** : l'écran affiche une invite de saisie, et chaque touche pressée sur le clavier matriciel est interceptée par `clavier.getKey()`. Les chiffres sont accumulés dans le tableau `codeSaisi` (limité à 4 caractères) et affichés sous forme d'astérisques pour la confidentialité. La touche * permet d'annuler la saisie, la touche # de la valider.
4. **Vérification du code (verifierCode)** : le code de référence est relu depuis l'EEPROM via `lireCodeEEPROM()`, puis comparé caractère par caractère au code saisi avec `strncmp()`. Cette approche évite de conserver le code de référence en permanence en RAM et illustre concrètement l'usage de la mémoire non volatile pour une base de données d'accès simple.
5. **Décision d'accès (accesAutorise, accesRefuse)** : en cas de succès, le servomoteur pivote vers la position *ouvert* pendant 3 secondes avant de se refermer automatiquement, la LED verte s'allume et un bip de confirmation retentit. En cas d'échec, la LED rouge clignote symboliquement (allumée en continu ici) et trois bips d'alerte successifs signalent le refus, sans action sur le servomoteur.
6. **Gestion de l'EEPROM (initialiserCodeEEPROM, lireCodeEEPROM)** : la bibliothèque `EEPROM.h` permet de lire et d'écrire des octets individuels à des adresses fixes (ici 0 à 3 pour les 4 chiffres du code). La vérification de validité du contenu existant (caractères entre '0' et '9') avant d'écrire le code par défaut évite de réinitialiser inutilement le code à chaque redémarrage, préservant ainsi la durée de vie limitée de la mémoire EEPROM.

Points forts du code :

- Vérification de la validité du contenu EEPROM avant écriture, évitant des cycles d'écriture inutiles et préservant la durée de vie de la mémoire.
- Masquage des chiffres saisis à l'écran (affichage d'astérisques), bonne pratique de sécurité même dans une version pédagogique simplifiée.
- Reverrouillage automatique du servomoteur après un délai fixe, évitant qu'un accès autorisé ne reste ouvert indéfiniment.
- Utilisation d'un *timeout* sur `pulseIn()`, évitant un blocage du programme en cas d'absence d'écho ultrasonique.

Pistes d'amélioration possibles :

- Intégrer un véritable historique des accès (horodatage, succès/échec) affiché sur l'écran ou stocké en EEPROM, conformément à la fonctionnalité prévue dans le cahier des charges.
- Ajouter un **chiffrement simple** (XOR avec une clé) du code stocké en EEPROM plutôt que de le conserver en clair, afin de limiter les risques en cas de lecture directe de la mémoire du microcontrôleur.

- Implémenter un compteur de tentatives échouées consécutives, avec verrouillage temporaire du clavier après un nombre défini d'essais incorrects (protection contre les attaques par force brute).
- Une fois le module RFID RC522 disponible, ajouter sa gestion via la bibliothèque MFRC522 et la communication SPI, en complément (et non en remplacement) du clavier, pour offrir deux méthodes d'authentification.

8 Ressources Complémentaires

- **Vidéo** – *RFID RC522 Arduino : Complete Access Control System*
<https://www.youtube.com/watch?v=hRhBbgwWYrU>
- **Vidéo** – *Contrôle d'accès Arduino RFID + Keypad + Servo (FR)*
<https://www.youtube.com/watch?v=hgR5s8lf3mA>

9 Conclusion

Ce projet de contrôle d'accès illustre l'articulation entre une interface de saisie utilisateur, une mémoire non volatile pour la persistance des données sensibles, et un actionneur mécanique simulant un verrou physique. La version Tinkercad, en remplaçant le module RFID par un capteur ultrason et l'écran OLED par un LCD I²C, permet de valider l'intégralité de la logique de contrôle d'accès (saisie, vérification EEPROM, déverrouillage temporisé, retours sonores et visuels). L'intégration du lecteur RFID RC522 via le bus SPI constituerait l'étape suivante naturelle pour obtenir un système d'authentification à deux facteurs (badge et/ou code), conforme au cahier des charges complet.